

Anexo técnico

Agente de Implantação Fiscal Sprint de IA · Grupo Assemble · Canal Sancon, implantação · junho a setembro de 2026

Este anexo acompanha o código-fonte publicado em <https://github.com/AldoBre/agente-copiloto-validacao-fiscal>. Ele existe para responder, sem a pressa de uma apresentação de dez minutos, três perguntas: **o que roda por trás, como se coloca isso de pé e por que cada escolha foi feita assim**.

Escopo deste documento. Aqui só se trata do **comparador de XML**: dois conjuntos de notas confrontados um contra o outro. O repositório contém outras frentes, que não fazem parte desta entrega e não são descritas aqui.

Sumário

- 1. O que a solução faz
- 2. A fronteira entre código e IA
- 3. Arquitetura
- 4. As tecnologias, e por que cada uma
- 5. Como rodar
- 6. Como usar
- 7. Testar sem dado real de cliente
- 8. Cobertura e limites conhecidos
- 9. Segurança e dado sensível
- 10. Deploy
- 11. Para outro canal reaproveitar

1. O que a solução faz

Numa virada de ERP, antes do go-live, o consultor precisa provar que a nota fiscal emitida pelo sistema novo bate com a que o cliente emitia no sistema antigo. Hoje isso é feito abrindo os dois XMLs lado a lado e comparando campo a campo.

A aplicação faz três coisas:

- 1. **Compara** os dois conjuntos de XML, campo a campo, e lista as divergências de imposto classificadas por severidade.
- 2. **Consolida** o lote: cem notas com a mesma divergência viram *uma* causa recorrente, porque uma causa é *um* ajuste de parametrização.
- 3. **Explica** cada divergência em texto, dizendo onde no ERP Senior aquilo se corrige, com a fonte citada.

O que ela **não** faz: não tem valor legal de validador fiscal, não substitui o julgamento do consultor e não garante que nenhuma divergência passou.

E há um limite de natureza, não de maturidade: **o critério de verdade é sempre o outro sistema**. O comparador responde "os dois lados divergem aqui". Ele não responde "este imposto está correto perante a lei". Se os dois sistemas estiverem errados do mesmo jeito, ele não acusa.

2. A fronteira entre código e IA

Esta é a decisão central do projeto, e tudo o mais decorre dela.

Etapa	Quem executa	Natureza
Leitura dos XMLs	Código	Determinístico
Comparação campo a campo	Código	Determinístico
Consolidação do lote em causas	Código	Determinístico
Explicação e orientação de correção	Modelo de IA	Probabilístico
Decisão de corrigir	Consultor	Humano

Por que a comparação não é feita por IA. Diferença entre dois campos de XML é um fato, não uma interpretação. Mesma entrada precisa produzir mesma saída, sempre, senão o resultado não é auditável e o consultor não pode confiar nele para assinar um go-live. O motor de comparação ([apps/comparador/services/comparador.py](#)) é 100% determinístico e não chama modelo nenhum.

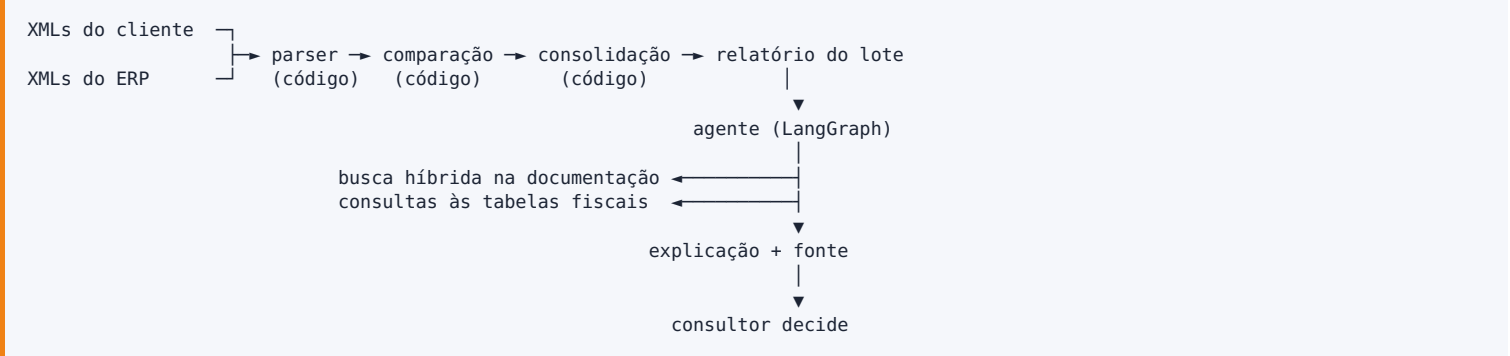
Por que existe IA, então. O que o código produz é uma lista de campos divergentes. Isso não diz ao consultor *onde arrumar*. Traduzir " `ICMS.CST` mudou de `00` para `20` "

em 87 notas" para "a regra de tributação desse grupo de produtos está apontando para redução de base de cálculo; confira a parametrização do grupo tributário" é trabalho de linguagem, e é aí que o modelo entra.

Por que o modelo não inventa alíquota. Ele não responde de memória. Toda afirmação numérica sai de uma consulta a tabela oficial vigente, exposta ao modelo como ferramenta (seção 3.3). Se a tabela não tem a resposta, a saída é "não verificado", não um palpite.

3. Arquitetura

3.1 O caminho do dado



3.2 Os módulos

App	Responsabilidade
comparador	Parser de NF-e/NFC-e e NFS-e, aritmética fiscal, pareamento de notas, comparação campo a campo, relatório individual e de lote
fiscal	Tabelas oficiais (NCM, TIPI, CEST, CFOP, FCP) que dão a base para classificar a divergência e ancorar a explicação
conhecimento	Raspagem da documentação Senior, divisão em trechos, embeddings, busca e regras de parametrização
provedores	Cadastro multi-IA e criptografia das chaves
agente	O grafo do agente, as ferramentas e a API de streaming
core	As telas: painel do comparador e configurações

3.3 O agente

Grafo em LangGraph (`apps/agente/graph.py`):



- **preparar** carrega o relatório da comparação anexada e extrai os campos divergentes;
- **recuperar** busca na documentação *antes* de o modelo ler a pergunta, o que garante que o ciclo termina;
- **responder** transmite a resposta token a token;
- **ferramentas** executa as consultas que o modelo pediu.

As ferramentas que o modelo usa ao explicar uma divergência. A regra por trás delas é uma só: afirmação factual sai de consulta a fonte oficial, nunca da memória do modelo.

Ferramenta	Fonte oficial
consultar_ncm	Portal Único Siscomex
consultar_ipi	TIPI vigente
consultar_cest	Convênio ICMS 142/2018
consultar_cfop	Tabela de CFOP aceitos na NF-e
consultar_fcp	Fundo de Combate à Pobreza por UF
aliquota_icms_interestadual	Resolução do Senado
buscar_documentacao	Base de conhecimento indexada

3.4 A busca na documentação

Busca híbrida, com três rankings fundidos por *Reciprocal Rank Fusion*:

- **BM25** sobre um índice invertido em memória, com stemming de português;
- **full-text search nativo do PostgreSQL**, que traz o stemming oficial e escala pelo índice GIN conforme a base cresce;
- **similaridade vetorial** via pgvector, quando há provedor de embeddings cadastrado.

O RRF combina rankings de escalas diferentes sem precisar normalizar score. Sem provedor de embeddings, o sistema continua funcionando com os dois rankings textuais: a busca fica pior, não quebra.

O índice BM25 é construído uma vez e reaproveitado. A versão ingênua, que tokeniza todos os trechos a cada pergunta, foi medida em 281 ms com 540 trechos, o que projeta cerca de 10 segundos com 20 mil trechos, e deixaria o chat inutilizável.

4. As tecnologias, e por que cada uma

Python 3.12

Não foi uma escolha de preferência: o ecossistema de agentes (LangChain, LangGraph, os SDKs de todos os provedores) é primariamente Python, e as bibliotecas de leitura de XML fiscal maduras também. Qualquer outra linguagem custaria escrever ponte.

Django 5.2 (com DRF) para a aplicação

Alternativa descartada: um front-end SPA com API separada. **Por quê:** o entregável precisa de tela de cadastro, upload, listagem, histórico e um `/admin` para editar regra de parametrização. Django entrega tudo isso sem construir nada. Uma SPA teria dobrado o trabalho de front para ganhar uma fluidez que o caso de uso não pede: o consultor sobe um lote e lê um relatório.

FastAPI montado dentro do Django, em `/api/ia`

Alternativa descartada: streaming pelo próprio Django. **Por quê:** o chat transmite a resposta token a token por SSE, e isso pede ASGI assíncrono de verdade. Em vez de migrar a aplicação inteira, o FastAPI é montado como sub-app só para essa rota. Existe um caminho síncrono de fallback (`chat_síncrono`) para quando o ambiente não suporta ASGI.

LangChain e LangGraph para o agente

Alternativa descartada: chamar a API do provedor direto, sem framework. **Por quê:** o LangGraph torna o fluxo um grafo explícito, com os nós nomeados. Isso importa menos pela conveniência e mais pela **auditabilidade**: dá para apontar no desenho onde o modelo pode agir e onde não pode, que é exatamente a pergunta que a diretoria e o time fiscal fazem. Chamada direta teria produzido a mesma funcionalidade com a fronteira implícita no meio do código.

O LangChain entra pela camada de conectores: trocar de provedor é trocar uma classe, não reescrever a integração.

Quatro provedores de IA suportados

Anthropic, OpenAI, Google Gemini e Azure AI Foundry. O Foundry é o usado em produção.

Alternativa descartada: fixar um provedor. **Por quê:** cada canal do grupo já tem contrato com alguém, e política de dado varia. Amarrar a solução a um fornecedor transformaria um ativo reaproveitável em um projeto de um canal só. Os conectores são importados sob demanda: instala quem vai usar.

O Azure AI Foundry merece nota à parte: são os mesmos modelos da OpenAI, mas hospedados na assinatura Azure do grupo. **O dado não sai do nosso tenant**, e esse é o motivo de ele existir num produto que lê nota fiscal de cliente.

O `requirements.txt` traz conectores de outros provedores (Grok, Mistral, DeepSeek, Ollama) porque o LangChain os oferece, mas eles **não estão implementados**: acrescentar um exige entrada no enum `Provedor` , no catálogo e na fábrica de clientes.

PostgreSQL 18 com pgvector

Alternativa descartada: um banco vetorial dedicado. **Por quê:** o embedding vira uma coluna `vector` na mesma tabela do trecho, e a similaridade roda no banco. Isso elimina uma peça de infraestrutura, um ponto de falha e um custo, e evita carregar a base inteira na RAM da aplicação. Para o volume de documentação de um ERP, o pgvector resolve com folga.

O SQLite continua funcionando (`DB_ENGINE=sqlite`) para quem quiser rodar sem infraestrutura nenhuma, mas sem busca vetorial.

Criptografia das chaves com Fernet

A chave de API do provedor é criptografada antes de ir para o banco, com chave mestra em `APP_ENCRYPTION_KEY` , fora do código. Sem isso, um dump de banco entregaria credencial de terceiro.

lxml para leitura de XML

XML fiscal é contrato: o leiaute é publicado, versionado e estável. Ler com parser é mais rápido, mais barato e mais confiável do que qualquer abordagem que envolva modelo de linguagem lendo o arquivo.

Tailwind via CDN e JavaScript sem framework

Por quê: são duas telas. Introduzir build de front, bundler e dependência de node para duas telas seria custo sem retorno, e mais uma coisa para outro canal instalar antes de rodar.

5. Como rodar

Requisitos

- Python 3.12
- Docker (só para o banco)

Passo a passo

```
cd agente_implantacao

python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

cp .env.example .env      # ajuste se quiser
docker compose up -d      # PostgreSQL em 127.0.0.1:5435
./run.sh
```

Abra <http://127.0.0.1:8000>.

O `run.sh` aplica as migrações, importa as tabelas fiscais oficiais, semeia as regras de parametrização, importa a base de conhecimento versionada e sobe o servidor em modo ASGI (que é o necessário para o streaming do chat funcionar). Para escolher host e porta: `./run.sh 0.0.0.0 8080`.

Rodar sem Docker

Define `DB_ENGINE=sqlite` no `.env`. Usa o arquivo `db.sqlite3` local. Serve para desenvolvimento e demonstração; não serve para uso real, e desliga a busca vetorial.

Endereços

Tela	Caminho
Painel: chat do agente e comparador	/
Comparação de lote	/lotes/
Comparação nota a nota	/comparar/
Configurações	/configuracoes/
Admin do Django	/admin/
Documentação da API de IA	/api/ia/docs

Configurar a IA

O chat exige um provedor cadastrado em **Configurações > Provedores de IA**. Um provedor de **chat** é obrigatório. Um de **embeddings** é opcional: sem ele, a busca na base de conhecimento usa só os rankings textuais.

Comandos de manutenção

Comando	Para quê
<code>importar_tabelas_fiscais</code>	Carrega ou atualiza NCM, TIPI, CEST, CFOP, FCP e a lista nacional de serviços
<code>importar_markdown</code>	Importa a base de conhecimento versionada no repositório
<code>ingerir_url</code> / <code>ingerir_links</code> / <code>ingerirZendesk</code>	Raspa documentação nova, a partir de URL, de uma lista de links ou do Zendesk
<code>reindexar</code>	Reconstrói os embeddings depois de trocar o provedor
<code>seed_regras</code>	Semeia as regras de parametrização a partir do mapa de campos
<code>gerar_exemplos</code> / <code>gerar_lote_exemplo</code>	Gera XMLs de teste com divergências plantadas
<code>exportar_feedback</code>	Exporta o feedback dos consultores sobre as respostas

6. Como usar

Comparação em lote, que é o caso principal

1. **Configurações › Provedores de IA:** cadastre a chave do provedor de chat.
2. **Lotes › Novo lote:** envie dois ZIPs, um com os XMLs do sistema atual do cliente e outro com os XMLs gerados pelo ERP Senior. Também aceita as duas pastas.
3. A aplicação **pareia** as notas entre os dois conjuntos. Nota sem par é reportada como tal, não descartada em silêncio.
4. O relatório sai agrupado por **causa recorrente**, ordenado por severidade e por número de notas afetadas. Cada causa traz as variações de valor e os produtos envolvidos.
5. **Abra o chat** a partir do relatório e pergunte sobre a causa. O agente já recebe o relatório como contexto: não é preciso descrever o problema.

Severidades

Severidade	Significado
Crítica	Muda o valor do imposto devido
Alta	Muda a classificação tributária, com efeito provável em imposto
Média	Diverge em campo que afeta apuração ou obrigação acessória
Baixa	Diferença cadastral ou de preenchimento, sem efeito de imposto

As três situações de cada campo

- **Conforme:** os dois lados batem, dentro da tolerância configurada.
- **Divergente:** os dois lados diferem.
- **Não verificado:** não há base oficial para afirmar. É deliberado. O sistema prefere declarar que não sabe a fingir que conferiu.

As tolerâncias são configuráveis por `COMPARADOR_TOLERANCIA_MONETARIA` e `COMPARADOR_TOLERANCIA_PERCENTUAL` , porque arredondamento de centavo entre dois sistemas não é divergência fiscal.

7. Testar sem dado real de cliente

```
python manage.py gerar_lote_exemplo --quantidade 100      # dois ZIPs, comparação em massa
python manage.py gerar_exemplos                          # dois XMLs, nota a nota
```

Os arquivos vão para `exemplos/` , já com divergências plantadas. É o caminho recomendado para demonstração e para o primeiro contato de outro canal: não expõe nota fiscal de cliente nenhum.

8. Cobertura e limites conhecidos

Documentos que o comparador lê

Documento	Situação
NF-e e NFC-e 4.00	Cobertura completa, item a item
NFS-e padrão nacional (<code>infNFSe</code> / <code>infDPS</code>)	Suporte básico: lê os grupos de imposto e retenções, com cobertura de campos menor que a da NF-e
Outros leiautes de NFS-e (ABRASf, prefeituras)	Não lidos nesta versão. O parser recusa o arquivo dizendo qual raiz esperava, em vez de tentar adivinhar

Limites que o código assume de propósito

O **critério de verdade é o outro sistema, não a lei**. É o limite mais importante e o menos óbvio. O comparador confronta dois XMLs e aponta onde eles divergem. Ele não confere uma nota sozinha contra a legislação vigente, e por isso não acusa erro em que os dois sistemas concordam. Dizer isso na frente do cliente é o que impede a ferramenta de ser vendida como algo que ela não é.

"Não verificado" é uma resposta legítima. Quando não há base para afirmar, porque a NFS-e nacional, de cobertura básica, não trouxe o campo ou porque a nota não tem par no outro conjunto, o resultado é "não verificado" e não "conforme". Silêncio tratado como conformidade é o pior defeito possível numa ferramenta de conferência.

Tolerância de arredondamento. Diferença de centavo entre dois sistemas não é divergência fiscal. As tolerâncias são configuráveis por `COMPARADOR_TOLERANCIA_MONETARIA` e `COMPARADOR_TOLERANCIA_PERCENTUAL` , e o escopo dos campos comparados por `COMPARADOR_ESCOPO` .

Pareamento entre os dois conjuntos. As notas são pareadas automaticamente, mas numeração divergente entre os sistemas acontece. Nota sem par é reportada como tal, nunca descartada em silêncio, e o lote pode ser reparado manualmente em `/lotes/<id>/repartear/` .

A explicação é probabilística. A lista de divergências é sempre a mesma para a mesma entrada. O texto que explica onde corrigir, não: é gerado por modelo. Por isso ele cita a fonte, e por isso a decisão de corrigir é do consultor.

9. Segurança e dado sensível

- **Chaves de API** criptografadas com Fernet antes de ir para o banco. A chave mestra vive em `APP_ENCRYPTION_KEY` , fora do código.
- **Banco de desenvolvimento preso ao loopback** no `docker-compose.yml` . Banco que guarda nota fiscal de cliente não deve aceitar conexão da rede.
- **Extração de ZIP protegida** contra travessia de caminho (`apps/comparador/services/arquivos.py`).
- **Nota fiscal sai da rede** quando o provedor de IA é externo. Para canal com restrição de dado, o Azure AI Foundry mantém a chamada dentro da assinatura Azure do grupo, em vez de sair para a API pública do provedor.

10. Deploy

A aplicação roda como container (`Dockerfile`): o entryptpoint aplica as migrações, semeia as regras de parametrização e sobe o servidor ASGI. Ao lado, um PostgreSQL com a extensão pgvector. Em produção usamos um serviço gerenciado de containers com banco gerenciado; a infraestrutura como código e os pipelines ficaram fora desta cópia.

A versão do PostgreSQL local acompanha a de produção de propósito, para não descobrir incompatibilidade só na migração.

11. Para outro canal reaproveitar

Item	O que é preciso
Infraestrutura	Docker e PostgreSQL. Roda numa máquina só, ou em qualquer serviço de containers
Provedor de IA	Uma chave de um dos quatro provedores. Chat obrigatório, embeddings opcional
Base de conhecimento	A documentação do módulo em markdown. O projeto já traz a raspagem da documentação Senior
Dado de entrada	Os XMLs dos dois lados: sistema atual do cliente e ERP em implantação
Esforço	Instalação em menos de um dia. O trabalho real é cadastrar o provedor e indexar a documentação do canal

O que se reaproveita mesmo fora do fiscal

O código fiscal serve para quem faz virada de ERP. O que serve para qualquer canal é **o desenho**:

1. A parte que produz o número é código determinístico e auditável.
2. A IA entra só na camada de linguagem, explicando o que o código achou.
3. Toda afirmação factual do modelo sai de consulta a fonte oficial, exposta como ferramenta, e não da memória do modelo.
4. Quando não há fonte, a saída é "não verificado", e não um palpite.
5. A decisão final é sempre de uma pessoa.

Esse padrão é transferível para conferência de folha, conferência de estoque, validação de cadastro, e qualquer outro processo em que hoje alguém compara duas telas na mão.